

EliteRestaurant Code Review Report

This report reviews the repository with a code-review lens rather than an architecture lens. The goal is to identify weak code, over-engineering, fragile design, stale or broken implementation, security gaps, and places where the code should be made stronger before the system grows further.

Review scope: `EliteRestaurantPro` , `EliteRestaurant.Api` , `SeedRunner` , and tooling under `Tools` .

Method: static code review plus targeted build verification where helpful.

Important note: findings are prioritized toward correctness, maintainability, security, and operational risk, not cosmetic style.

Findings Summary

Severity	Count	Main themes
Critical	2	Broken tooling and architecture that can fail builds.
High	4	Credential handling, weak API auth boundaries, secret storage, and schema/runtime coupling.
Medium	5	Overgrown classes, duplicated logic, missing retention, partial-save patterns, and lack of tests.

Findings

1. `ClearActiveOrders` is broken code, not just weak code

Location: `Tools/ClearActiveOrders/Program.cs`

```
var path = AppDbContext.DatabasePath
var removed = AppDbContext.DeleteAllActiveOrders()
Console.WriteLine($"Database: {path}")

Build verification:
error CS0117: 'AppDbContext' does not contain a definition for 'DatabasePath'
```

The tool references `AppDbContext.DatabasePath` , but the active `AppDbContext` no longer defines that member. A targeted build verification failed with `CS0117: 'AppDbContext' does not contain a definition for 'DatabasePath'` .

Why this matters: this is dead operational tooling that looks valid until someone tries to use it. It also confirms that old SQLite-era assumptions are still present in the repository. For maintenance software, broken code is worse than ugly code because it creates false confidence during incidents.

Stronger code direction: either remove this utility entirely or update it to the real PostgreSQL-first runtime model and keep it in the normal build path.

2. The API is coupled to a WPF executable project, and that coupling is already causing build fragility

Location: EliteRestaurant.Api/EliteRestaurant.Api.csproj ,
EliteRestaurantPro/EliteRestaurantPro.csproj

```
<!-- EliteRestaurant.Api.csproj -->
<ItemGroup>
  <ProjectReference Include="..\EliteRestaurantPro\EliteRestaurantPro.csproj" />
</ItemGroup>

<!-- EliteRestaurantPro.csproj -->
<OutputType>WinExe</OutputType>
<TargetFramework>net8.0-windows</TargetFramework>
<UseWPF>true</UseWPF>

<BuildVerification>
  <Error CS0103: The name 'InitializeComponent' does not exist in the current context />
</BuildVerification>
```

The API project has a direct `ProjectReference` to `EliteRestaurantPro.csproj`, which is a WPF Windows executable project with `<UseWPF>true</UseWPF>` and `<OutputType>WinExe</OutputType>`. In this environment, building `EliteRestaurant.Api.csproj` failed while the referenced WPF temporary project was being built, producing WPF-generated-code errors such as missing `InitializeComponent` and a missing suitable static `Main`.

Why this matters: a web API should not depend directly on a desktop UI application project. Even when it happens to work, it makes backend builds, CI, and deployment much more brittle and harder to port. The current failure shows this is already a real risk, not just a theoretical architecture complaint.

Stronger code direction: move shared entities, DB context, and domain utilities into a dedicated class library, then let both the API and WPF application reference that library instead of each other.

3. Employee PINs are stored, compared, and even displayed as plaintext

Location: EliteRestaurantPro/Models/Employee.cs ,
EliteRestaurant.Api/Security/TabletAuthService.cs ,
EliteRestaurantPro/ViewModels/StaffLoginViewModel.cs ,
EliteRestaurantPro/Views/EmployeesView.xaml

```

public class Employee
{
    public string PinCode { get; set; } = string.Empty;

    // API login
    Where(e => string.Equals((e.PinCode ?? string.Empty).Trim(), normalizedPin,
        StringComparison.Ordinal))

    // Desktop login
    Where(e => e.PinCode.Trim() == pin)

    // Employee list UI
    TextBlock Text="{Binding PinCode, StringFormat=PIN {0}}"/>

```

The system stores `PinCode` directly on the employee model, compares it directly during login, and shows it in the employee UI using text such as `PIN {0}`. The employee form also edits PIN values directly.

Why this matters: any database dump, local workstation access, screenshot, or admin view leaks live staff credentials. This is one of the clearest places where the code needs stronger security practices, even for an internal network product.

Stronger code direction: hash PINs, stop rendering them in the employee list, and replace direct comparison with secure verification.

4. The API authentication surface is intentionally weak and leaks tokens into URLs

Location: `EliteRestaurant.Api/Program.cs`,
`EliteRestaurant.Api/Security/TabletAuthService.cs`,
`EliteRestaurant.Api/Controllers/ServerPortalController.cs`,
`EliteRestaurant.Api/Security/HttpAuthExtensions.cs`

```

// COR:
options.AddPolicy("LanOnly", policy =>
    policy.AllowAnyHeader().AllowAnyMethod().AllowAnyOrigin());

// In-memory token session
private readonly ConcurrentDictionary<string, AuthenticatedStaffSession> sessions
    = new(StringComparer.Ordinal);
var token = Guid.NewGuid().ToString("N");

// Token embedded into URLs:
var token = Uri.EscapeDataString(session.Token);
var logoUrl = $"/api/server/assets/restaurant-logo?token={token}";
var employeePhotoUrl = $"/api/server/assets/me-photo?token={token}";

// Custom bearer parsing
return header[prefix.Length..].Trim();

```

Several design choices stack together here:

- `Program.cs` sets CORS to `AllowAnyHeader().AllowAnyMethod().AllowAnyOrigin()`.
- Sessions are stored only in a singleton in-memory `ConcurrentDictionary`.
- `/api/server/config` generates asset URLs that include the access token as a query string parameter.
- Token validation is custom and bypasses the standard ASP.NET authentication/authorization pipeline.

Why this matters: permissive CORS plus query-string token usage increases the chance of accidental token leakage through logs, browser history, screenshots, referrers, or diagnostics. In-memory sessions also disappear on restart, which is an availability and UX weakness.

Stronger code direction: move assets behind standard bearer-header auth, tighten CORS to expected origins, and consider standard auth middleware or at least a more formal token/session mechanism.

5. Database connection secrets are stored in plaintext and surfaced in settings UI

Location: `EliteRestaurantPro/Utils/AppSettings.cs` ,
`EliteRestaurantPro/Utils/SettingsManager.cs` ,
`EliteRestaurantPro/ViewModels/AppearanceSettingsViewModel.cs` ,
`EliteRestaurantPro/App.xaml.cs`

```
public sealed class DatabaseSettings
{
    public string Provider { get; set; } = "PostgreSql";
    public string PostgreSQLConnectionString { get; set; } = string.Empty;

    DatabaseConnectionString = db.PostgreSqlConnectionString ?? string.Empty;
    settings.Database.PostgreSqlConnectionString = DatabaseConnectionString.Trim();
    SettingsManager.Save( settings);
}
```

The PostgreSQL connection string is saved directly as `Database.PostgreSqlConnectionString` in the local app settings file, loaded directly into editable UI state, and saved back without any protection or redaction.

Why this matters: the database secret is effectively stored as workstation-level plaintext configuration. Any local user, support backup, or copied profile can expose the database credentials. This is especially sensitive when the database is shared beyond a single local machine.

Stronger code direction: prefer environment variables or OS-protected secret storage for credentials, and avoid showing the full connection string in editable UI by default.

6. `AppDbContext` mixes too many responsibilities and uses startup-time schema mutation instead of migrations

Location: EliteRestaurantPro/Data/AppDbContext.cs

```
public static void Initialize()
{
    using var db = new AppDbContext();
    db.Database.EnsureCreated();
    EnsureReservationsSchema(db);
    EnsureOrderReservationColumns(db);
    EnsureSharedDraftSchema(db);
    EnsureHistoricalActivity(db, days: 14);
    FinancialTransactionService.EnsureCompletedOrderRevenues(db);
}

private static void EnsureSharedDraftSchema(AppDbContext db)
{
    db.Database.ExecuteSqlRaw("""
        CREATE TABLE IF NOT EXISTS "SharedOrderDrafts" (
            ...
        )
    """);
}
```

`AppDbContext` currently acts as:

- Entity Framework model definition
- Startup schema creator
- Schema patch runner via raw SQL
- Legacy SQLite migration/import engine
- Sample/bootstrap data loader
- Consistency-repair utility
- Active-order cleanup helper

It also uses `Database.EnsureCreated()` plus manual `ExecuteSqlRaw` schema patches instead of EF migrations.

Why this matters: this is over-engineered in one dimension and under-structured in another. Too much unrelated logic lives in one file, while schema evolution remains ad hoc. This increases regression risk whenever the database model changes.

Stronger code direction: split schema/migration logic, legacy import logic, repair logic, and operational helpers into distinct classes, and move toward real migrations.

7. `CreateOrderViewModel` is a god object and still contains archived duplicate implementations in the same file

Location: EliteRestaurantPro/ViewModels/CreateOrderViewModel.cs

```
public sealed class CreateOrderViewModel : AdminBaseViewModel
{
    ...
}

#if false
public sealed class CreateOrderViewModel : AdminBaseViewModel
{
    ...
}

using System.Collections.ObjectModel;
...
public sealed class CreateOrderViewModel : AdminBaseViewModel
{
    ...
}
```

The active class is already very large, and the file also contains duplicate historical versions of `CreateOrderViewModel` behind `#if false` blocks and archived code sections. The file is large enough that multiple offsets still reveal repeated class declarations.

Why this matters: this is classic over-engineering-by-accumulation. The file is trying to hold too many concerns at once: UI state, discounts, table flow, reservation integration, shared drafts, payment display, and tablet-specific logic. Keeping dead copies of previous implementations in the same file makes future bug-fixing and review harder.

Stronger code direction: extract focused services or helper classes for draft persistence, order submission, table loading, reservation binding, and totals; remove archived code from source control instead of keeping it under preprocessor blocks.

8. Shared drafts grow without update or retention logic

Location: EliteRestaurantPro/Utils/SharedOrderDraftStore.cs

```
public static IReadOnlyList<SharedDraftRow> ListServerDrafts(int employeeId)
{
    return db.SharedOrderDrafts
        .Where(d => d.EmployeeId == employeeId && d.Portal == ServerPortal)
        .OrderByDescending(d => d.UpdatedAtUtc)
        .Take(50)
        .ToList();
}
```

```

public static SharedDraftRow SaveServerDraft(...)
{
    var entity = new SharedOrderDraft
    {
        UniqueId = Guid.NewGuid().ToString("N"),
        // ...
    };
    db.SharedOrderDrafts.Add(entity);
    db.SaveChanges();
}

```

Every draft save inserts a brand new `SharedOrderDraft` row with a new GUID. Retrieval is capped to the latest 50 rows, but there is no update-in-place behavior, retention policy, cleanup job, or per-user cap enforced at write time.

Why this matters: usage grows the table forever while only hiding older rows from the UI. That is weak persistence hygiene and will become a maintenance issue if server staff use drafts heavily.

Stronger code direction: add update semantics, retention limits, or scheduled cleanup, and consider keeping only a bounded number of drafts per employee.

9. Login and role-resolution logic is duplicated across desktop and API

Location: `EliteRestaurant.Api/Security/TabletAuthService.cs` ,
`EliteRestaurantPro/ViewModels/StaffLoginViewModel.cs`

```

// API
if (string.Equals(portal, "KitchenBar", StringComparison.OrdinalIgnoreCase))
{
    return candidates.FirstOrDefault(e => IsKitchenBarRole(e.Role));
}

// Desktop
Employee? emp = Kind switch
{
    StaffPortalKind.Server => candidates.FirstOrDefault(e =>
        e.Role.Equals("Server", StringComparison.OrdinalIgnoreCase)),
    StaffPortalKind.KitchenBar => candidates.FirstOrDefault(e =>
        IsKitchenBarRole(e.Role)),
    _ => candidates.FirstOrDefault(e =>
        e.Role.Equals("Cashier", StringComparison.OrdinalIgnoreCase))
};

```

The desktop and API both independently implement active-employee lookup, PIN matching, and role filtering for server, cashier, and kitchen/bar roles. Even the kitchen/bar role matching list is duplicated.

Why this matters: duplicated auth logic drifts over time. One login surface can silently accept or reject cases that the other surface handles differently. Auth logic is exactly the kind of logic that should not fork.

Stronger code direction: centralize staff credential validation and role eligibility in a shared service that both UI and API consume.

10. Several write paths save in multiple steps without an explicit transaction boundary

Location: EliteRestaurant.Api/Controllers/ServerPortalController.cs ,
EliteRestaurantPro/ViewModels/CreateOrderViewModel.cs ,
EliteRestaurantPro/ViewModels/AdminOrdersViewModel.cs

```
// API
table.Status = "Occupied";
db.SaveChanges();
AppDbContext.ReconcileTableStatusesWithOrders(db);
db.SaveChanges();

// Desktop admin orders
order.Status = "Waiting";
db.SaveChanges();
AppDbContext.ReconcileTableStatusesWithOrders(db);
db.SaveChanges();
```

Several workflows follow a pattern like "save order changes, then reconcile table statuses, then save again." This is visible in multiple code paths where `db.SaveChanges()` is called, then `AppDbContext.ReconcileTableStatusesWithOrders(db)`, then `db.SaveChanges()` again.

Why this matters: if the second step fails, state can be partially committed. The application may recover later, but it is still weaker than treating the workflow as one unit of work.

Stronger code direction: wrap multi-step state transitions in transactions or reshape the logic so the full change is persisted atomically.

11. The repository has no automated test coverage

Location: repository-wide observation

```
Repository scan result:
result of search in 'C:\Users\ernes\Documents\EliteRestaurant': 0 files found

Pattern used:
**/*test*
```

No test projects or obvious test files were found in the repository scan.

Why this matters: this codebase contains payroll rules, order-status workflows, financial posting logic, table occupancy rules, reservation state rules, and draft persistence. Those are all areas where silent regression is expensive. The lack of tests increases the cost of every refactor.

Stronger code direction: start with focused tests around payroll calculation, order workflow transitions, totals calculation, and table occupancy reconciliation.

12. Widespread `AsEnumerable()` usage weakens query efficiency and hides translation problems instead of solving them

Location: `TabletAuthService`, `StaffLoginViewModel`, `EmployeesViewModel`, `ReservationsViewModel`, `AttendanceViewModel`, other ViewModels

```
// API auth
var candidates = db.Employees.AsNoTracking()
    .Where(e => e.EmploymentStatus == "Active")
    .AsEnumerable()
    .Where(e => string.Equals((e.PinCode ?? string.Empty).Trim(), normalizedPin,
StringComparison.Ordinal))
    .ToList();

// Desktop staff login
var candidates = db.Employees.AsNoTracking()
    .Where(e => e.EmploymentStatus == "Active")
    .AsEnumerable()
    .Where(e => e.PinCode.Trim() == pin)
    .ToList();
```

The code frequently switches from database-side filtering to in-memory filtering with `AsEnumerable()` because certain string operations or helper methods are not SQL-translatable.

Why this matters: for current small datasets this may be acceptable, but it is a weak long-term pattern. It bypasses indexes, obscures where performance boundaries are, and normalizes application-side filtering as the escape hatch.

Stronger code direction: normalize data so queryable comparisons are simpler, or move repeated query logic into dedicated database-safe helpers.

Over-Engineering Hotspots

Hotspot	Why it feels over-engineered or overgrown
<code>AppDbContext.cs</code>	One file tries to be schema manager, migration bridge, bootstrapper, repair engine, import tool, and persistence layer at the same time.
<code>CreateOrderViewModel.cs</code>	Large multi-concern ViewModel with retained historical copies instead of extracted smaller components.
API referencing WPF app	Shared reuse was pursued in the wrong place; a dedicated domain library would be simpler and stronger than reusing the full desktop app project.

Hotspot	Why it feels over-engineered or overgrown
Custom token/session/auth flow	Reimplements a lighter auth system, but now owns the burden of session cleanup, header parsing, role checks, and token distribution details.

Where the Code Should Be Stronger First

1. Separate shared business/data code into a neutral class library and remove the API-to-WPF dependency.
2. Fix or delete broken operational tools so the repository build reflects reality.
3. Stop storing and displaying PINs in plaintext; harden auth and secret handling.
4. Introduce EF migrations and reduce startup-time schema mutation.
5. Break up the largest ViewModels, starting with `CreateOrderViewModel`.
6. Add a first layer of regression tests around payroll, totals, order transitions, and table reconciliation.

Validation Notes

Check	Observed result
<code>dotnet build</code> <code>Tools/ClearActiveOrders/ClearActiveOrders.csproj</code>	Failed with missing <code>AppDbContext.DatabasePath</code> .
<code>dotnet build</code> <code>EliteRestaurant.Api/EliteRestaurant.Api.csproj</code>	Failed in this environment while building the referenced WPF project, reinforcing the layering/build-coupling concern.
<code>dotnet build</code> <code>EliteRestaurantPro/EliteRestaurantPro.csproj</code>	Succeeded, but only after a file-lock retry warning.

This report focuses on actionable engineering risk. Some findings are architectural, some are operational, and some are security-related. The recommended next step is to treat the top three findings as backlog items with explicit owners rather than as documentation-only observations.